

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Kolar Umme Sadiya	Reg. No.:	192425117
Section / Batch:	3 rd year / AI&MI	Date:	27/08/2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time monitoring system receives data from multiple sensors with different sampling frequencies and response-time requirements. Develop a pseudocode for an interrupt scheduling mechanism that assigns suitable interrupt priorities to the sensors based on their sampling rate, urgency, and data-processing requirements. The algorithm should manage simultaneous interrupt requests and ensure that time-critical sensor data is serviced without causing excessive CPU overhead.
2. A high-speed data acquisition system must transfer continuous streams of data between an I/O device and memory while preventing data loss when the incoming data rate exceeds the processing speed of the CPU. Write a pseudocode for a double-buffered DMA mechanism that alternates between two memory buffers during data transfer. The algorithm should detect buffer completion, switch buffers, generate appropriate interrupts, and handle buffer overflow conditions.
3. A computer system contains several I/O devices with different data transfer sizes and timing requirements. Design a pseudocode for an I/O scheduling algorithm that determines the order in which pending I/O requests should be serviced. The algorithm should consider device priority, request size, waiting time, latency requirements, and available bandwidth to improve overall I/O performance while preventing starvation.
4. A high-performance embedded system uses multiple DMA-capable devices that may request memory access at the same time. Develop a pseudocode for a DMA channel arbitration mechanism that allocates memory bus access among competing DMA requests. The algorithm should consider request priority, transfer size, urgency, and fairness, while ensuring that high-priority transfers are completed within their required time constraints.
5. A real-time system must communicate simultaneously with several devices that have different communication speeds and reliability requirements. Write a pseudocode for an adaptive I/O management mechanism that monitors device performance and dynamically changes the communication method or transfer parameters when congestion, delays, or errors are detected. The algorithm should monitor transfer status, detect performance degradation, adjust resource allocation, and maintain reliable communication with minimal CPU overhead.

Code of Conduct

I Kolar Umme Sadiya-192425117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kolar Umme Sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

1. Interrupt Scheduling Mechanism for a Real-Time Monitoring System

Problem Understanding

A real-time monitoring system receives data from multiple sensors. Each sensor may have a different sampling frequency, urgency, and data-processing requirement.

The main objectives are:

- Assign suitable priorities to different sensors.
- Handle simultaneous interrupt requests.
- Give higher priority to time-critical sensors.
- Reduce excessive CPU overhead.
- Ensure that important sensor data is processed within the required time.

The interrupt scheduling mechanism should consider **sampling rate, urgency, and processing requirement** when assigning priorities.

Proposed Solution

A priority-based interrupt scheduling mechanism is used.

Sensors with:

- Higher sampling frequency
- Higher urgency
- Greater processing requirements

are assigned higher priority.

The system maintains a priority queue for pending interrupt requests.

Pseudocode

ALGORITHM Interrupt_Scheduling

INPUT:

Sensor_List

Each sensor contains:

Sampling_Rate

Urgency

Processing_Requirement

OUTPUT:

Serviced interrupt requests

BEGIN

FOR each Sensor in Sensor_List DO

Calculate Priority_Score:

Priority_Score =

Sampling_Rate_Weight +

Urgency_Weight +

Processing_Weight

Assign Priority based on Priority_Score

END FOR

WHILE system is running DO

Check all sensors for interrupt requests

IF interrupt request is detected THEN

Add sensor request to Priority_Queue

END IF

IF multiple interrupt requests are present THEN

Sort requests according to Priority

END IF

IF Priority_Queue is not empty THEN

Select highest-priority request

IF request is time-critical THEN

Service interrupt immediately

ELSE

Service according to priority order

END IF

Process sensor data

Clear interrupt request

END IF

END WHILE

END

Working

1. The system monitors all sensors continuously.
2. The sampling rate, urgency, and processing requirement of each sensor are considered.
3. A priority score is calculated.
4. Higher-priority sensors are placed before lower-priority sensors.
5. When multiple interrupts occur simultaneously, the priority queue determines which interrupt should be serviced first.
6. Time-critical sensor interrupts receive immediate CPU attention.
7. After processing, the interrupt is cleared.
8. The CPU continues servicing the next pending interrupt.

Example

Consider three sensors:

Sensor	Sampling Rate	Urgency	Priority
Sensor A	High	High	Highest
Sensor B	Medium	Medium	Medium
Sensor C	Low	Low	Lowest

If all three generate interrupts at the same time:

Sensor A → Sensor B → Sensor C

The CPU services Sensor A first because it is the most time-critical.

Advantages

- Provides fast response to critical sensors.
- Handles simultaneous interrupts efficiently.
- Reduces unnecessary CPU overhead.
- Prevents important sensor data from being delayed.
- Supports real-time monitoring.

Conclusion

A priority-based interrupt scheduling mechanism is suitable for real-time monitoring systems because it ensures that time-critical sensor data is processed first while lower-priority requests are handled efficiently.

2. Double-Buffered DMA Mechanism for High-Speed Data Acquisition

Problem Understanding

A high-speed data acquisition system continuously transfers data from an I/O device to memory. If the incoming data rate is higher than the CPU processing speed, data loss may occur.

To prevent this problem, a **double-buffered DMA mechanism** is used.

Two memory buffers are used:

- Buffer A
- Buffer B

While DMA fills one buffer, the CPU can process the other buffer. When one buffer is full, DMA switches to the other buffer.

The system must detect buffer completion, switch buffers, generate interrupts, and handle overflow conditions.

Proposed Solution

The system uses two buffers alternately.

Buffer A → DMA Transfer → Processing

At the same time:

Buffer B → CPU Processing

When Buffer A becomes full, DMA switches to Buffer B and Buffer A becomes available for processing.

Pseudocode

ALGORITHM Double_Buffered_DMA

INPUT:

Continuous_Data_Stream

OUTPUT:

Data transferred without loss

BEGIN

Initialize Buffer_A

Initialize Buffer_B

Set Active_Buffer = Buffer_A

Set Processing_Buffer = Buffer_B

Start DMA Transfer

WHILE system is running DO

IF Active_Buffer becomes full THEN

Generate Buffer_Complete interrupt

Switch Active_Buffer

IF Active_Buffer == Buffer_A THEN

Processing_Buffer = Buffer_B

ELSE

Processing_Buffer = Buffer_A

END IF

Start DMA transfer into Active_Buffer

Process Processing_Buffer

Mark Processing_Buffer as FREE

END IF

IF Active_Buffer is full AND other buffer is NOT free THEN

Generate Buffer_Overflow alert

Stop or temporarily control incoming data

Handle overflow condition

END IF

END WHILE

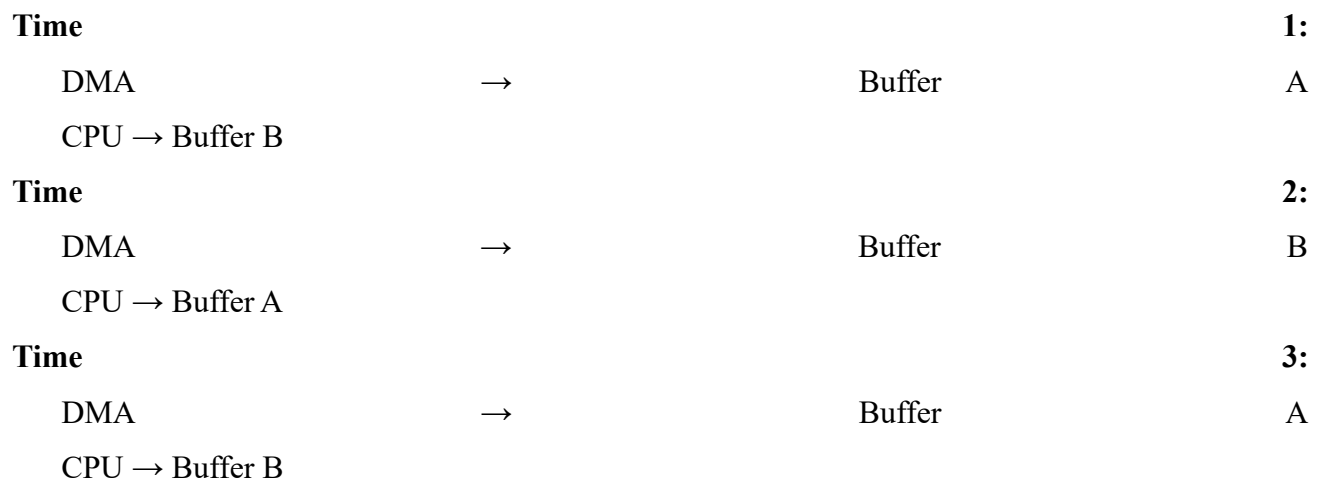
END

Working

1. Two buffers are initialized.
2. DMA starts transferring incoming data into Buffer A.
3. While DMA fills Buffer A, the CPU can process Buffer B.
4. When Buffer A becomes full, DMA generates an interrupt.
5. DMA switches to Buffer B.
6. Buffer A is given to the CPU for processing.
7. The CPU processes Buffer A while DMA fills Buffer B.
8. When Buffer B becomes full, the process is reversed.
9. The system continuously alternates between the two buffers.
10. If both buffers are unavailable, an overflow condition is generated.

Example

The process can be represented as:



This allows continuous data transfer and processing.

Advantages

- Reduces data loss.
- Supports continuous high-speed data transfer.
- Reduces CPU involvement.
- Improves DMA efficiency.
- Allows simultaneous data transfer and processing.
- Handles buffer completion automatically.

Conclusion

Double-buffered DMA is suitable for high-speed data acquisition because it allows one buffer to receive data while the other is being processed. This reduces data loss and improves overall throughput.

3. I/O Scheduling Algorithm

Problem Understanding

A computer system contains several I/O devices with different data transfer sizes and timing requirements. Multiple I/O requests may be waiting at the same time.

The scheduling algorithm must consider:

- Device priority
- Request size
- Waiting time
- Latency requirements
- Available bandwidth
- Starvation prevention

The objective is to determine the best order for servicing pending I/O requests.

Proposed Solution

A priority-based scheduling algorithm with aging is used.

A request receives a higher scheduling score when:

- Its device has high priority.
- It has been waiting for a long time.
- It has a strict latency requirement.
- Suitable bandwidth is available.

Aging is used to prevent starvation of low-priority requests.

Pseudocode

ALGORITHM IO_Scheduling

INPUT:

Pending_IO_Requests

OUTPUT:

Ordered list of serviced requests

BEGIN

WHILE Pending_IO_Requests is not empty DO

FOR each Request in Pending_IO_Requests DO

Calculate Waiting_Time

Calculate Priority_Score:

Priority_Score =

Device_Priority

+ Waiting_Time_Weight

+ Latency_Requirement

+ Bandwidth_Availability

+ Request_Size_Factor

END FOR

Select Request with highest Priority_Score

IF Request is close to its latency deadline THEN

Give Request immediate priority

END IF

Service selected Request

Remove completed Request from Pending_IO_Requests

Increase waiting priority of remaining requests

END WHILE

END

Working

1. All pending I/O requests are placed in a queue.
2. The system calculates a priority score for every request.
3. Device priority is considered.
4. Waiting time is considered using aging.
5. Requests with strict latency requirements receive higher priority.
6. Available bandwidth is considered before selecting a request.
7. The highest-scoring request is serviced.
8. After completion, it is removed from the queue.
9. The priority of waiting requests increases gradually.

10. This prevents starvation.

Example

Suppose three requests are waiting:

Request	Device Priority	Waiting Time	Latency Requirement
R1	High	Low	High
R2	Medium	High	Medium
R3	Low	Very High	Low

Initially, R1 may be serviced first because of its high priority and latency requirement.

However, R3's priority increases as its waiting time increases. Eventually, R3 receives service, preventing starvation.

Advantages

- Improves I/O performance.
- Reduces waiting time.
- Gives priority to critical requests.
- Prevents starvation.
- Uses available bandwidth efficiently.
- Supports different device requirements.

Conclusion

The proposed I/O scheduling algorithm combines priority, waiting time, request size, latency, and bandwidth. Aging ensures that low-priority requests are eventually serviced, preventing starvation.

4. DMA Channel Arbitration Mechanism

Problem Understanding

A high-performance embedded system contains multiple DMA-capable devices. Several devices may request memory bus access simultaneously.

The arbitration mechanism must consider:

- Request priority
- Transfer size
- Urgency
- Fairness
- Required completion time

High-priority transfers must be completed within their required time constraints.

Proposed Solution

A priority-based DMA arbitration mechanism with fairness is used.

Each DMA request receives a score based on:

- Priority
- Urgency
- Transfer size
- Waiting time

Waiting time is included to prevent starvation.

Pseudocode

ALGORITHM DMA_Channel_Arbitration

INPUT:

DMA_Request_List

OUTPUT:

Allocated DMA channel

BEGIN

WHILE DMA requests are pending DO

FOR each DMA_Request DO

Calculate Priority_Score:

Priority_Score =
Device_Priority
+ Urgency
+ Waiting_Time
+ Transfer_Size_Factor

END FOR

Check for requests with critical deadlines

IF critical request exists THEN

Select highest-priority critical request

ELSE

Select request with highest Priority_Score

END IF

Allocate Memory_Bus to selected request

Perform DMA transfer

IF transfer is completed THEN

Release Memory_Bus

Remove request from pending list

END IF

Increase waiting priority of remaining requests

END WHILE

END

Working

1. DMA-capable devices send memory access requests.
2. The arbitration unit collects all pending requests.

3. Each request is assigned a priority score.
4. Critical and urgent requests are identified.
5. A high-priority urgent request receives the memory bus first.
6. The DMA transfer is performed.
7. After completion, the bus is released.
8. Waiting requests receive increased priority over time.
9. This ensures fairness and prevents starvation.

Example

Suppose three DMA devices request access:

- **DMA 1:** High priority, urgent
- **DMA 2:** Medium priority
- **DMA 3:** Low priority but waiting for a long time

DMA 1 receives the bus first because it is urgent.

After DMA 1 completes, DMA 2 may receive access.

If DMA 3 continues waiting, its aging value increases, allowing it to eventually receive the bus.

Advantages

- Gives priority to urgent transfers.
- Improves bus utilization.
- Prevents starvation.
- Supports fairness.
- Meets real-time deadlines.
- Efficiently manages multiple DMA devices.

Conclusion

A priority-based DMA arbitration mechanism with aging provides an effective balance between urgency and fairness. It ensures that critical transfers are completed on time while allowing lower-priority devices to access the memory bus eventually.

5. Adaptive I/O Management Mechanism

Problem Understanding

A real-time system communicates with multiple devices that have different communication speeds and reliability requirements.

The system must:

- Monitor device performance.
- Detect congestion.
- Detect delays.
- Detect communication errors.
- Dynamically change communication methods.
- Adjust transfer parameters.
- Maintain reliable communication.
- Minimize CPU overhead.

Proposed Solution

An adaptive I/O management mechanism continuously monitors each device.

Depending on the device condition, the system can:

- Increase or decrease transfer size.
- Change DMA parameters.
- Change interrupt frequency.
- Allocate additional bandwidth.
- Retry failed transfers.
- Reduce transfer rate during congestion.
- Give additional resources to reliable high-speed devices.

Pseudocode

ALGORITHM Adaptive_IO_Management

INPUT:

Device_List

OUTPUT:

Reliable and optimized I/O communication

BEGIN

Initialize communication parameters

WHILE system is running DO

FOR each Device in Device_List DO

Monitor:

Transfer_Speed

Error_Rate

Waiting_Time

Bandwidth_Usage

Queue_Length

IF Error_Rate is HIGH THEN

Retry failed transfer

Reduce Transfer_Size

Increase Error_Recovery_Level

ELSE IF Queue_Length is HIGH OR congestion is detected THEN

Reduce Transfer_Rate

Adjust DMA parameters

Allocate resources according to priority

ELSE IF Transfer_Speed is LOW THEN

Increase available resources if possible

Optimize Transfer_Size

ELSE

Maintain current communication parameters

END IF

END FOR

Update device performance statistics

END WHILE

END

Working

1. The system initializes communication parameters for all devices.
2. Device performance is continuously monitored.
3. Transfer speed, error rate, waiting time, bandwidth usage, and queue length are measured.
4. If a high error rate is detected, failed transfers are retried.
5. Transfer size can be reduced to improve reliability.
6. If congestion is detected, the transfer rate is adjusted.
7. If a device is performing slowly, resources can be reallocated.
8. If communication is stable, the existing parameters are maintained.
9. The system continuously updates device performance information.
10. The process repeats throughout system operation.

Example

Suppose Device A normally transfers data at high speed but suddenly experiences congestion.

The system detects:

High Queue Length → Congestion Detected

The adaptive mechanism can:

Reduce Transfer Size → Adjust DMA Parameters → Reduce Congestion

If Device B starts producing errors:

High Error Rate → Retry Transfer → Reduce Transfer Size → Improve Reliability

Advantages

- Adapts automatically to changing conditions.
- Reduces communication errors.
- Handles congestion.
- Improves bandwidth utilization.
- Reduces CPU overhead.
- Maintains reliable communication.
- Suitable for real-time systems.

Conclusion

Adaptive I/O management provides a flexible solution for systems containing devices with different communication speeds and reliability requirements. By continuously monitoring performance and dynamically adjusting transfer parameters, the system can maintain reliable communication while minimizing CPU overhead.

OVERALL CONCLUSION

The five pseudocode algorithms demonstrate different approaches for designing efficient I/O communication mechanisms.

1. **Interrupt Scheduling** assigns priorities to sensor interrupts according to sampling rate, urgency, and processing requirements.
2. **Double-Buffered DMA** allows continuous data transfer while another buffer is being processed.
3. **I/O Scheduling** selects pending requests based on priority, waiting time, latency, request size, and bandwidth.
4. **DMA Arbitration** manages simultaneous DMA requests while maintaining priority and fairness.
5. **Adaptive I/O Management** dynamically changes communication parameters according to congestion, delays, and errors.

These mechanisms help improve **CPU utilization, latency, throughput, bandwidth utilization, interrupt response, data transfer efficiency, and overall system reliability.**

Therefore, selecting the appropriate I/O mechanism based on system requirements is essential for developing high-performance and reliable computer systems.